# 1. 프로그램 로직 데이터 구조 (Data Model)

각 영웅 데이터를 아래와 같이 3개 레이어(**기본 스탯**, **태그/시너지**, **스킬/카운터 특성**)로 수치화하여 DB(JSON 형식 등)로 관리합니다.

JSON

```
{
  "id": "julius",
  "name": "율리우스",
  "faction": "Way of Progress", // 진영 시너지
  "role": "Tank",
  "stats": {
    "physical_attack": 45000,
    "armor": 28000,
    "magic_defense": 22000,
    "crit_chance": 0
  },
  "tags": ["Shield", "Buff_Crit", "Progress"],
  "counters": {
    "hard_against": ["debuff_heavy"], // 디버프 위주 덱에 강함
    "vulnerable_to": ["iris", "amira"]  // 아이리스, 아미라에 약함
  }
}
```

# 2. 덱 평가 알고리즘 설계 (핵심 3대 평가 요소)

프로그램이 5인 덱을 입력받았을 때 총점을 계산하는 로직입니다.

## ① 스탯 및 실체 데미지 공식 (Nexters 공식 반영)

게임 내 실제 전투 공식을 로직으로 작성하여 물리/마법 감소율을 계산합니다.

- **물리 피해 감소율 (DpD_{p}Dp):** 1−11+Armor−ArmorPen30001 - \frac{1}{1 + \frac{\text{Armor} - \text{ArmorPen}}{3000}}1−1+3000Armor−ArmorPen1
- **마법 피해 감소율 (DmD_{m}Dm):** 1−11+MagicDef−MagicPen30001 - \frac{1}{1 + \frac{\text{MagicDef} - \text{MagicPen}}{3000}}1−1+3000MagicDef−MagicPen1
- **치명타 확률 (CCC):** CritChanceCritChance+TargetMainStat×100%\frac{\text{CritChance}}{\text{CritChance} + \text{TargetMainStat}} \times 100%CritChance+TargetMainStatCritChance×100%

> **프로그램 반영:** 공격자의 관통력(Penetration)과 방어자의 방어력(Armor/Magic Def) 차이를 계산해 **"해당 덱의 실제 방어 무력화 성능"**을 점수화합니다.

### ② 시너지 점수 (Synergy Score)

아군 5명의 스킬 버프가 서로 얼마나 겹치는지 판단합니다.

- **진영 시너지:** 같은 진영(진보의 길, 자연의 길, 혼돈의 길 등) 영웅 수 ×\times× 가중치
- **스탯 버프 시너지:**
  - 예: 케일라(치명타 딜러) + 트리스탄(물리관통 버프) + 세바스찬(치명타 극대화) = **치명 딜링 시너지 점수 +30**
  - 예: 율리우스 + 저지 + 아이작 = **진보 쉴드/속도 시너지 점수 +40**

### ③ 카운터 & 마이너스 요소 (Counter & Negative Factors)

상대 덱과의 상성을 비교하여 상쇄 점수를 산출합니다.

- **하드 카운터 (Hard Counter):** 상대 핵심 기믹을 아예 무력화할 때
  - 상대에 **마법 딜러(셀레스티, 오리온 등)** 존재 ↔\leftrightarrow↔ 내 덱에 **아이작(Isaac)** 합류 시 마법 덱 상성 점수 **+50 (하드 카운터)**
  - 상대에 **크리티컬 덱** 존재 ↔\leftrightarrow↔ 내 덱에 **헬리오스(Helios)** 합류 시 **+40**
  - 상대에 **회피 덱(단테, 옥타비아 등)** 존재 ↔\leftrightarrow↔ 내 덱에 **코르넬리우스/하이디** 등 대응 기믹 미포함 시 **-30 (마이너스 요인)**
- **소프트 카운터 (Soft Counter):** 힐러 차단(요르겐, 셀레스티 등) 스킬 보유 여부에 따른 감점/가점

## 3. 파이썬(Python) 기반 핵심 판정 로직 예시

파이썬 코드 구조로 매칭 결과를 출력하는 알고리즘 예시입니다.

Python

```
class DeckAnalyzer:
    def __init__(self, my_deck, enemy_deck):
        self.my_deck = my_deck
        self.enemy_deck = enemy_deck

    def analyze_synergy(self):
        score = 0
        tags = [tag for hero in self.my_deck for tag in hero['tags']]

        # 진보의 길 쉴드 메타 시너지 검사
        if tags.count('Progress') >= 3 and 'Shield' in tags:
            score += 35

        # 물리 치명타 시너지 검사
        if 'Buff_Crit' in tags and 'Physical_DPS' in tags:
            score += 25

        return score

    def analyze_counter_matchup(self):
        counter_score = 0
        enemy_ids = [h['id'] for h in self.enemy_deck]
        my_ids = [h['id'] for h in self.my_deck]

        # 하드 카운터 검사 (예: 상대 마법덱 vs 아이작)
        is_enemy_magic = any('Magic_DPS' in h['tags'] for h in self.enemy_deck)
        if is_enemy_magic and 'isaac' in my_ids:
            counter_score += 40  # 아이작 상성 우위

        # 헬리오스 vs 크리티컬 덱
        is_enemy_crit = any('Buff_Crit' in h['tags'] for h in self.enemy_deck)
        if is_enemy_crit and 'helios' in my_ids:
            counter_score += 40

        return counter_score

    def get_result(self):
        synergy = self.analyze_synergy()
        counter = self.analyze_counter_matchup()
        total = synergy + counter

        win_rate_est = min(max(50 + (total * 0.5), 10), 95)
        return {
            "시너지 점수": synergy,
            "상성 우위 점수": counter,
            "예상 승률": f"{win_rate_est}%"
        }
```

## 4. 구체적 프로그램 구현 단계 추천

1. **데이터베이스 구축 (JSON / SQLite):**
   - 게임 내 주요 영웅 30~40명의 **주요 스탯, 스킬 특성(디버프, 힐, 관통, 방어 등), 태그**를 로컬 DB로 정리합니다.
2. **시너지/상성 룰북(Rules Engine) 정의:**
   - "특정 영웅 A가 상대 덱 B를 만났을 때 가중치" 테이블(Counter Matrix) 작성.
3. **GUI / 웹 인터페이스 구성:**
   - **Streamlit**이나 **PyQt / React**를 활용해, 5명의 아군 영웅과 5명의 적군 영웅을 클릭해 선택하면 승률 및 시너지 리포트가 출력되도록 구현합니다.

직접 데이터를 수집하여 파이썬이나 웹 기반 시뮬레이터(매칭 판정기) 형태로 개발해 보시는 것을 추천합니다!